

Practical: 2

Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:

* Algorithm:-

1) Linear Search:-
 Input: An array
 Output: Target Element
 Step 1:- Start
 Step 2:- Read the array and the target element t
 Step 3:- Set $i=0$
 Step 4:- Compare $array[i]$ with t
 Step 5:- If $array[i] == t$, return the index.
 Step 6:- Otherwise, increment i and repeat step 4, until the array ends
 Step 7:- If element not found then return -1
 Step 8:- Stop

2) Binary Search:-
 Input: Sorted array
 Output: Target Element
 Step 1:- Start
 Step 2:- Read a sorted array and target element t
 Step 3:- Set $l=0$ and $h=n-1$
 Step 4:- Calculate the middle position

$$m = (l+h)/2$$

 Step 5:- If $arr[m] == t$ then return m
 Step 6:- If $arr[m] < t$, set $l=m+1$; else set $h=m-1$;
 Step 7:- Repeat while $(l \leq h)$
 Step 8:- If not found return -1
 Step 9:-

Program:

```
#include <iostream>
using namespace std;

// Linear Search
int linear(int array[], int s, int t) {
    for (int i = 0; i < s; i++) {
        if (array[i] == t) {
            return i;
        }
    }
    return -1;
}

// Binary Search
int binary(int array[], int s, int t) {
    int l = 0;
    int h = s - 1;

    while (l <= h) {
        int m = (l + h) / 2;

        if (array[m] == t) {
            return m;
        }
        else if (array[m] < t) {
            l = m + 1;
        }
        else {
            h = m - 1;
        }
    }

    return -1;
}

void display(int array[], int n) {
    cout << "Array is:\n";

    for (int i = 0; i < n; i++) {
        cout << array[i] << " ";
    }

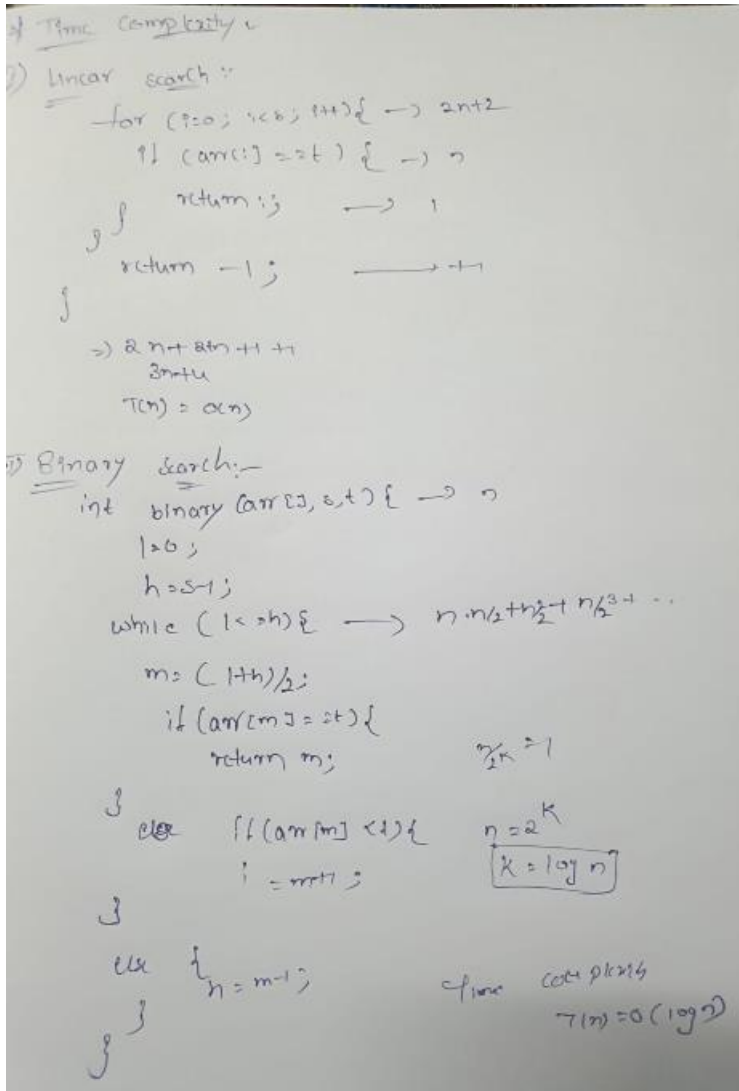
    cout << endl;
}
```

```
int main() {  
  
    int array[] = {2, 6, 7, 9, 11};  
    int n = sizeof(array) / sizeof(array[0]);  
  
    int t = 6;  
  
    display(array, n);  
  
    int f = linear(array, n, t);  
    int g = binary(array, n, t);  
  
    cout << "\nTarget element: " << t << endl;  
  
    if (f != -1) {  
        cout << "Linear Search: Element found at index " << f << endl;  
    }  
    else {  
        cout << "Linear Search: Element not found" << endl;  
    }  
  
    if (g != -1) {  
        cout << "Binary Search: Element found at index " << g << endl;  
    }  
    else {  
        cout << "Binary Search: Element not found" << endl;  
    }  
  
    return 0;  
}
```

Output:

```
Array is:  
4 5 3 7 1  
  
Target element: 7  
Linear Search: Element found at index 3  
Binary Search: Element found at index 3  
  
=== Code Execution Successful ===
```

Time Complexity:



Conclusion:

the Linear Search and Binary Search algorithms were successfully implemented and their time complexities were analyzed. Linear Search checks each element sequentially and can be used on both sorted and unsorted arrays, with a time complexity of **$O(n)$** in the worst case. Binary Search is more efficient, but it requires the array to be sorted. It repeatedly divides the search space into half and has a time complexity of **$O(\log n)$** in the worst case. Therefore, Binary Search is more efficient than Linear Search for searching elements in large sorted datasets.